



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .

2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the actual set of all distinct positions (x, y) the agent could occupy on the grid, along with the legal transitions between them, it's a fixed graph defined by the grid dimensions and walls. The search tree is what the algorithm builds while searching: a tree of nodes representing paths explored from the start. The same state can appear in multiple places in the tree (different paths reaching it), so the search tree is typically much larger than the state space, potentially infinite if cycles aren't handled, whereas the state space itself is finite and fixed.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set solves the repeated-state problem: without it, the algorithm would treat the search as a Tree Search, where the same grid cell could be re-added to the frontier every time it's reached via a different path. Since the grid has cycles (you can walk in a loop around a wall), a DFS agent without reached could expand the same cell indefinitely, going back and forth forever and never terminating, or at best doing enormous redundant work. reached converts it into a Graph Search by ensuring each cell is expanded at most once, guaranteeing termination.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS explores level by level (shallowest nodes first via the FIFO queue), so the first time it reaches the goal, it does so via the shortest possible path in terms of number of steps. Since every move in this grid has equal cost (step cost = 1), shortest path = optimal path, so BFS is guaranteed optimal.

DFS explores as deep as possible down one branch before backtracking (LIFO stack). It commits to whichever neighbor happens to be expanded first (in this code, the order comes from DIR_VECTORS iteration) and follows that path all the way, only backing up when it hits a dead end. This produces long, winding routes because DFS has no notion of "distance so far", it finds a path to the goal, not the shortest one.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS: space complexity is $O(b^d)$, where the frontier holds all nodes at the current depth level before moving to the next. On a huge grid, the frontier can balloon to contain a huge fraction of all reachable cells at once (imagine the "wavefront" expanding outward) this is the real bottleneck: exponential/very large frontier size in the worst case, even though the branching factor here is only 4.

DFS: space complexity is only $O(b \cdot d)$, it only needs to store the current path (and siblings not yet explored) down to the deepest node, i.e., linear in the depth of the tree. So DFS is far more memory-efficient on a huge grid.

The trade-off: BFS's reached set and frontier can consume memory proportional to the number of cells visited so far (which grows large fast on $1000 \times 1000 = 1,000,000$ cells), while DFS's frontier stays proportional to the path length, so BFS is the one likely to run out of memory first, while DFS remains lean but may find far longer, suboptimal paths.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING